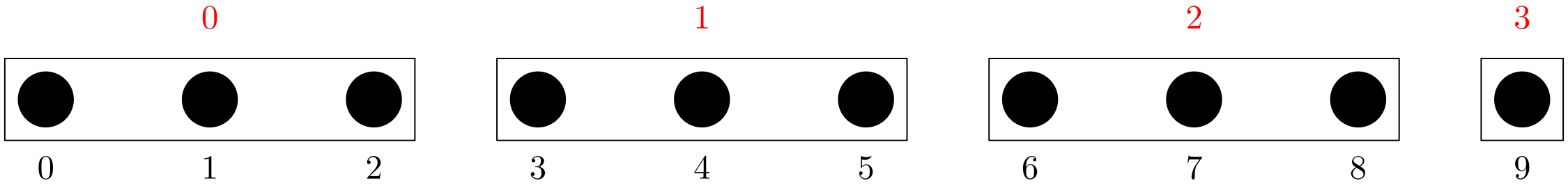
从分块到线段树

在线处理区间询问

序列分块

n 个东西从左到右排成一行，编号 0 到 $n-1$，就是一个**序列**。

选择一个正整数 $1 \le B \le n$，把这个序列每 B 个分成一**块**（block）。称 B 为**块长**。



$n = 10$，$B = 3$

序列里编号 i 的元素所在的块的编号是 $\lfloor i/B \rfloor$。

数列分块入门 1

给你一个长为 n 的整数序列 $a = (a_1, \ldots, a_n)$。处理 n 个操作。操作有两类：

- `0 l r c`：把 $a_l, \ldots, a_r$ 每个都加上 c。
- `1 l r c`：询问 a_r 的值。（忽略 l 和 c）

限制

- $1 \le n \le 300000$
- a_i 的初始值，c，每次修改过后的 a_i 都在 long long 范围内。

取 $B = \lfloor \sqrt{n} \rfloor$，对序列 a 进行分块。

如果一次区间加操作会把第 i 块里的每个数都加上 c，那么我们并不去修改这个块里数，而是把这个操作记录下来。

具体地，我们对第 i 块维护一个变量 `add[i]`，表示区间加操作对这一块整体加的数的总和。

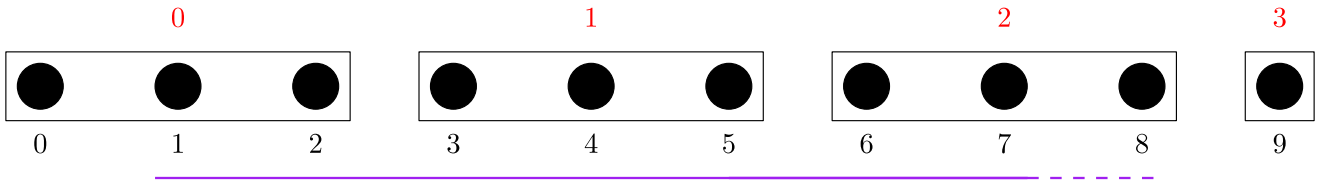
如果一次区间加操作只涉及第 i 块里的一部分元素，那么我们就去逐个修改相关的元素。
注意到这样的块在区间两端，最多两个，需要修改的元素至多 $2(B-1)$ 个。

第 j 个数的真实值就是 `a[j] + add[j / B]`。

处理区间加操作的时间是 $O(\sqrt{n})$，回答询问的时间是 $O(1)$。

```cpp
int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    int n;
    cin >> n;
    vector<long long> a(n);
    for (int i = 0; i < n; i++)
        cin >> a[i];
    int B = (int) sqrt(n);
    int NB = (n + B - 1) / B;
    vector<long long> add(NB);
    int q = n;
    while (q--) {
        int type, l, r; long long c;
        cin >> type >> l >> r >> c;
        if (type == 0) {
            --l;
            int lb = (l + B - 1) / B;
            int rb = r / B;
            if (lb > rb)
                for (int i = l; i < r; i++) {
                    a[i] += c;
                }
            else {
                for (int i = l; i < lb * B; i++)
                    a[i] += c;
                for (int i = lb; i < rb; i++)
                    add[i] += c;
                for (int i = rb * B; i < r; i++)
                    a[i] += c;
            }
        } else {
            r--;
            cout << a[r] + add[r / B] << '\n';
        }
    }
}
```

- 序列下标总是从 0 开始。
- 总是用**左闭右开**的方式表示区间。
- `NB` 是块数。



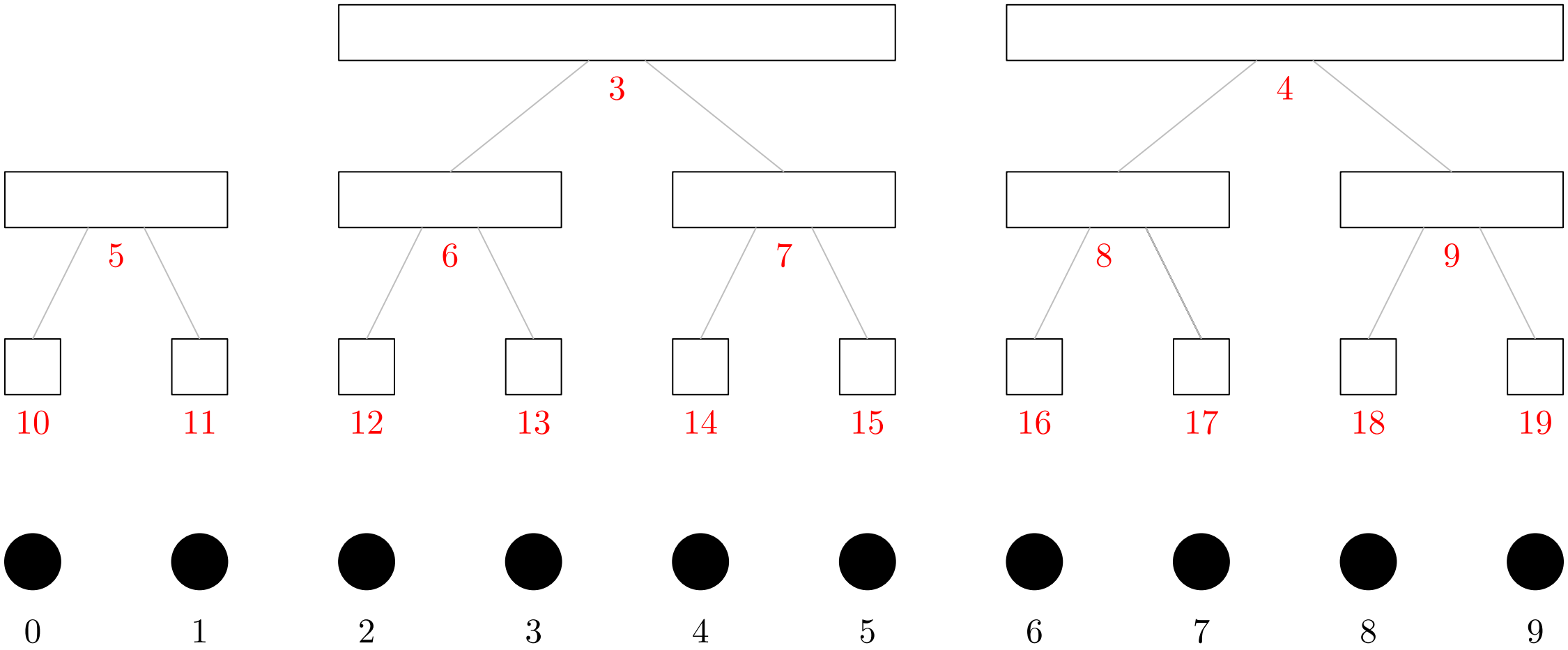
`l` $= 1$，`r` $= 8$。

`lb` $= 1$，`rb` $= 2$。

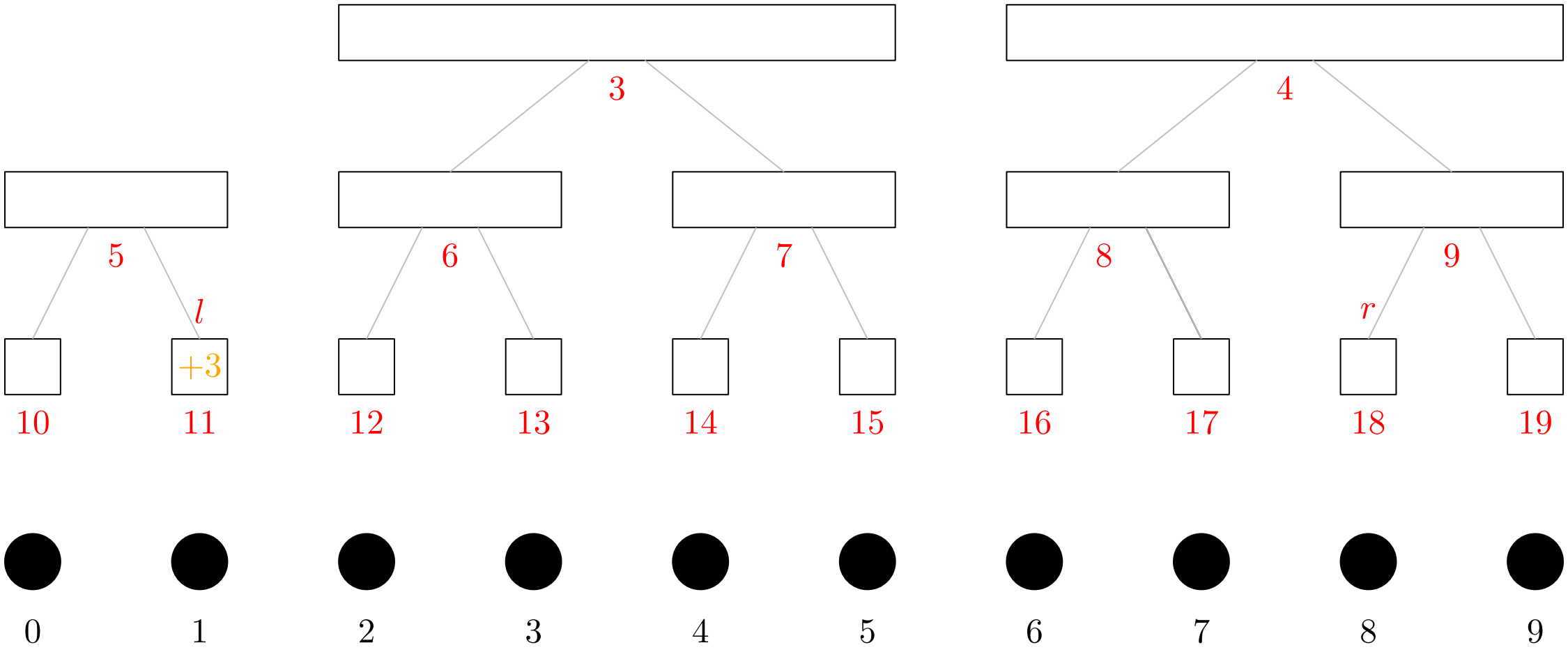
对于区间 $[1, 8)$，1 是**整块**，0 和 2 是**边块**。

一般的，[`lb` , `rb`) 是整块，`lb` - 1 和 `rb` 是边块。

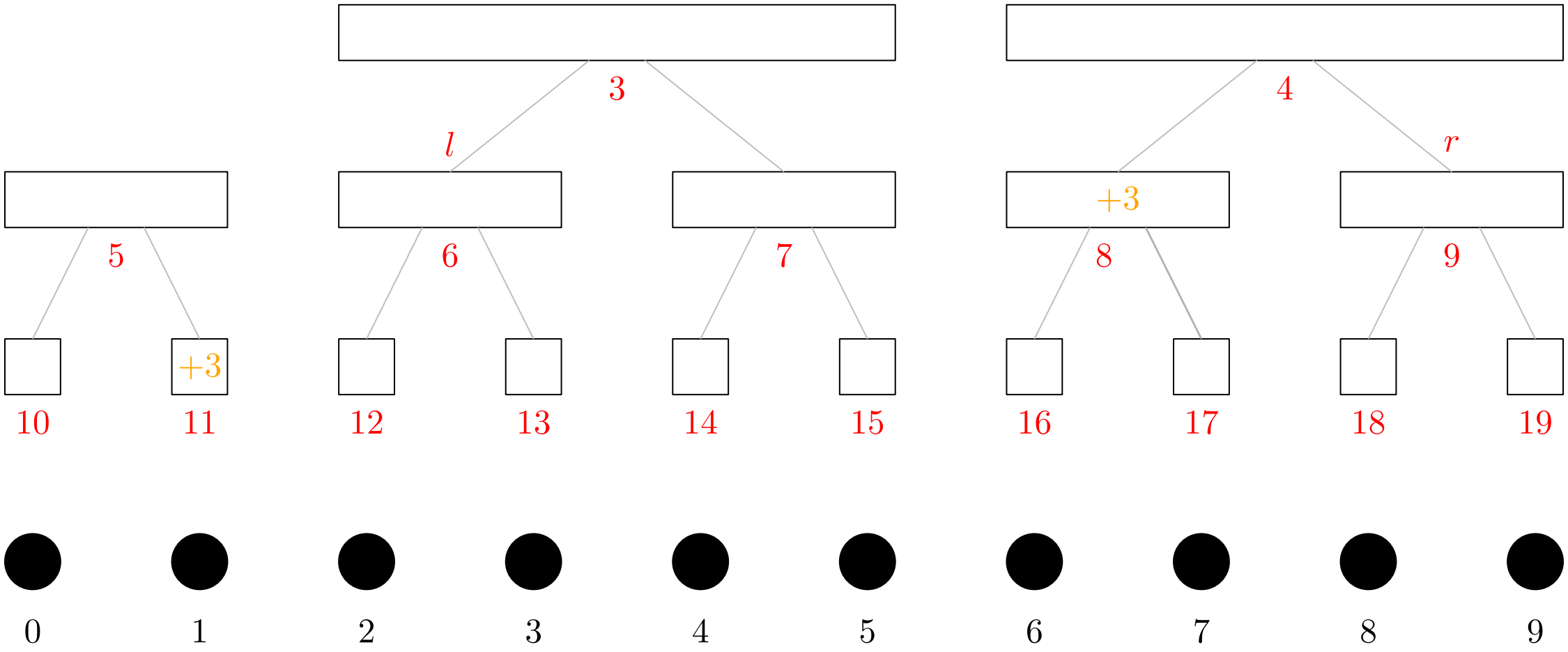
线段树：多层次分块



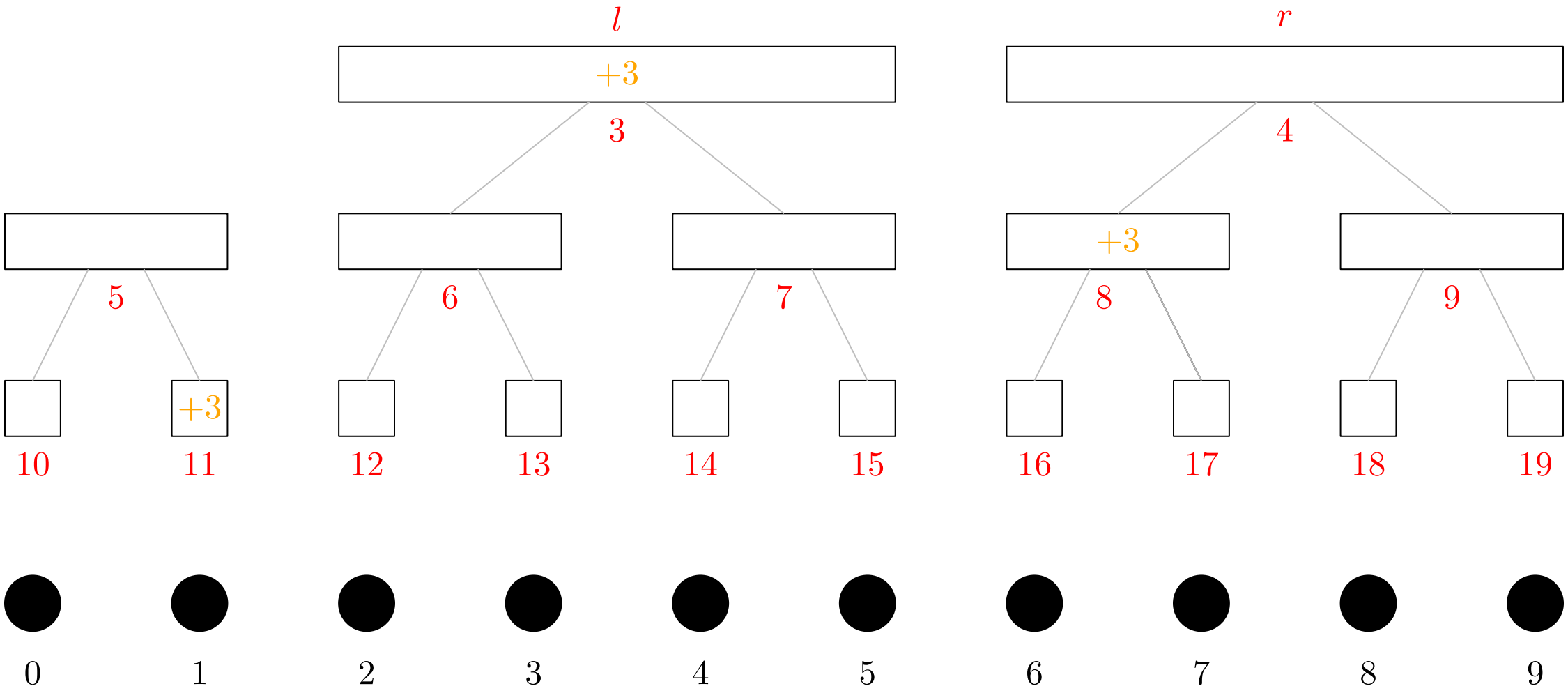
给 a_1 到 a_7 加 3



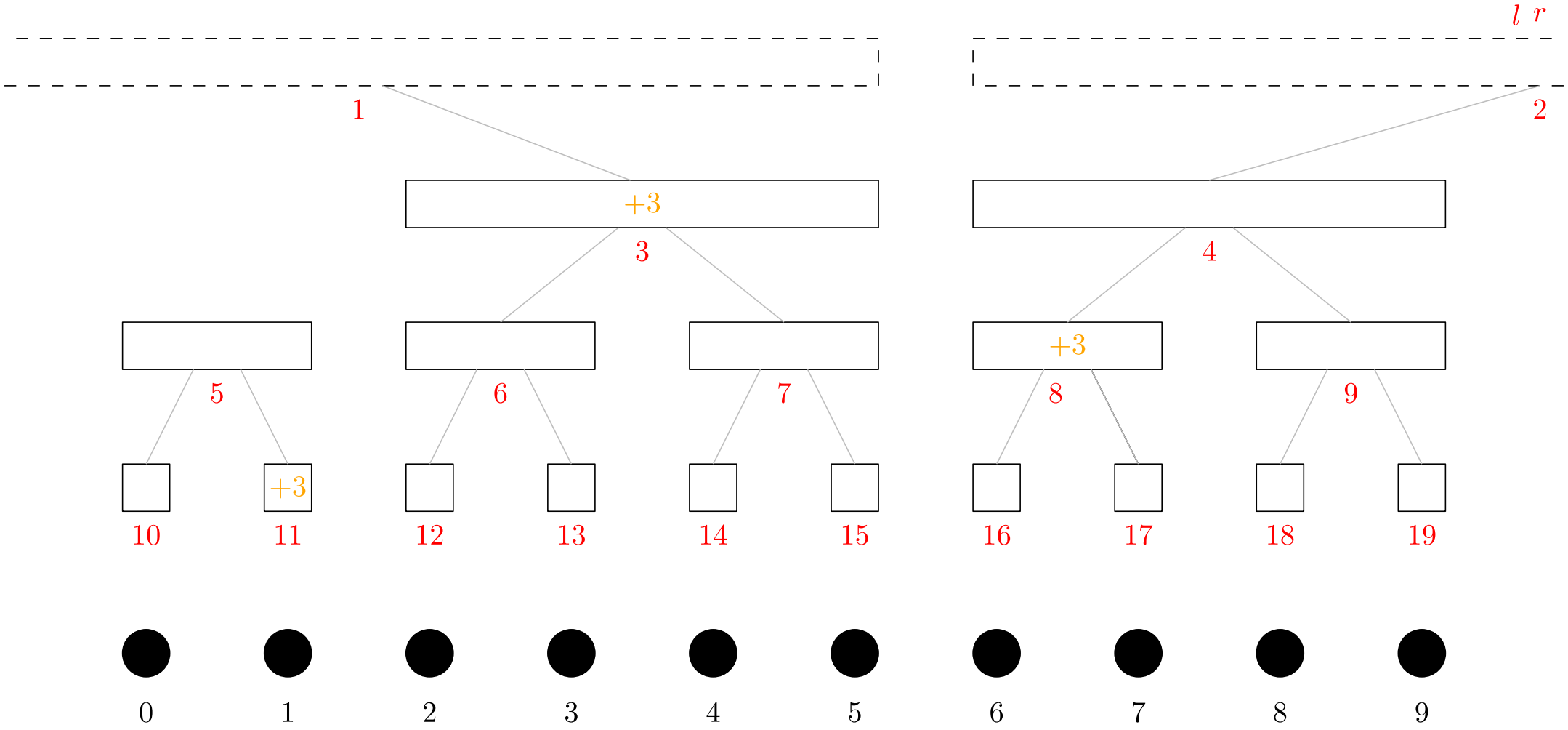
给 a_1 到 a_7 加 3



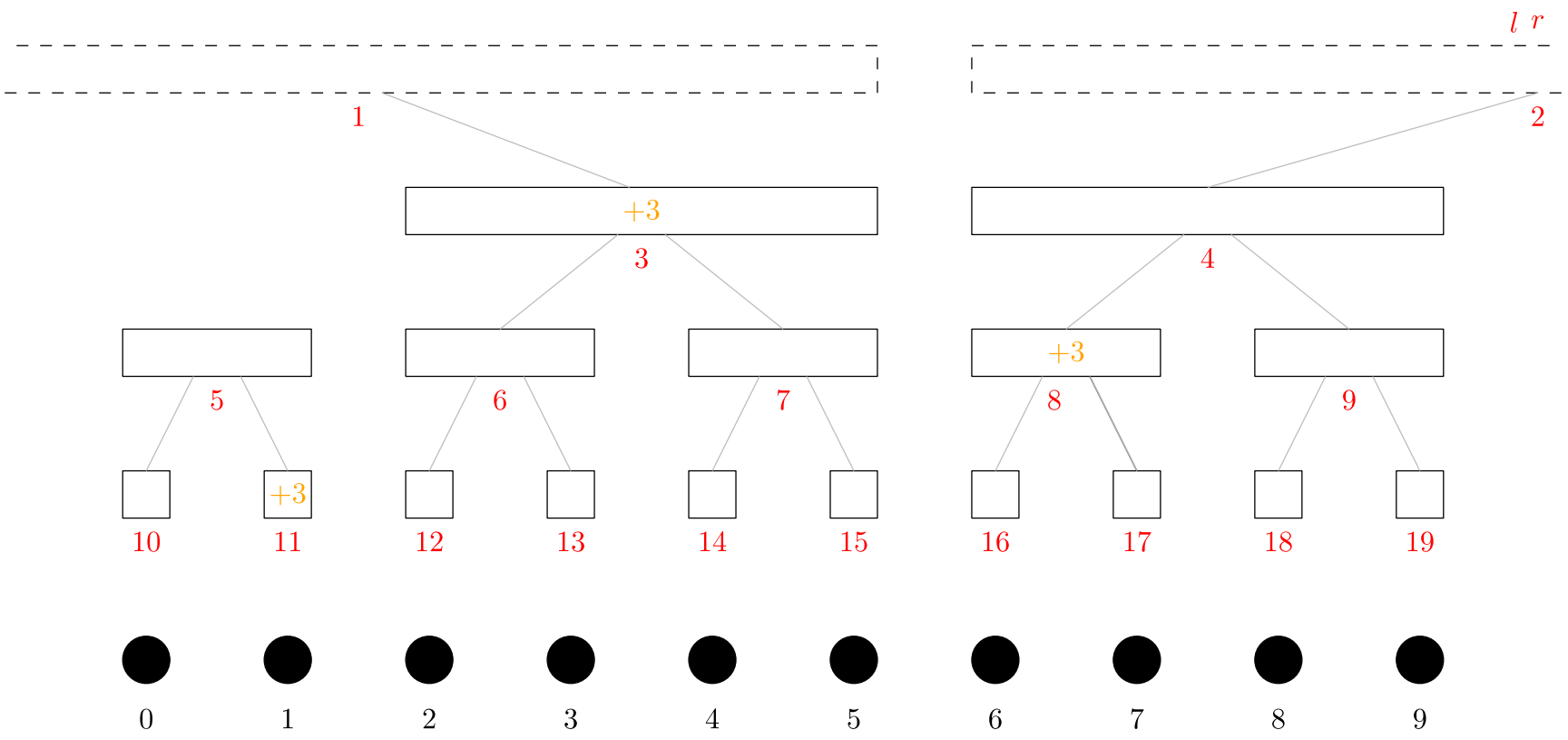
给 a_1 到 a_7 加 3



给 a_1 到 a_7 加 3



给 a_1 到 a_7 加 3



对于区间 $[1, 8)$ 来说，11、8、3 是整块，但是它们的父节点不是整块（而是边块），我们把这样的整块称为**极大整块**。

```cpp
int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    int n;
    cin >> n;
    vector<long long> a(2 * n);
    for (int i = 0; i < n; i++)
        cin >> a[i + n];

    int q = n;
    while (q--) {
        int type, l, r; long long c;
        cin >> type >> l >> r >> c;
        if (type == 0) {
            l--;
            l += n;
            r += n;
            while (l < r) {
                if (l & 1) a[l++] += c;
                if (r & 1) a[--r] += c;
                l /= 2;
                r /= 2;
            }
        } else {
            r--;
            r += n;
            long long ans = 0;
            while (r > 0) {
                ans += a[r];
                r /= 2;
            }
            cout << ans << '\n';
        }
    }
}
```

- 处理区间加操作的时间：$O(\log n)$
- 回答询问的时间：$O(\log n)$

数列分块入门 2

给你一个长为 n 的整数序列 $a = (a_1, \ldots, a_n)$。处理 n 个操作。操作有两类：

- `0 l r c`：把 $a_l, \ldots, a_r$ 每个都加上 c。
- `1 l r c`：询问 $a_l, \ldots, a_r$ 中小于 c^2 的数的个数。

限制

- $1 \le n \le 200000$
- a_i 的初始值，c 在 int 范围内。

分块

取 $B = \lfloor \sqrt{n} \rfloor$。

- 对每个块 i，维护对块 i 的整体加的操作所加的数的总和 `add[i]`。
- 维护一个长为 n 的数组 `a`。
  对于每个 $j = 0, \ldots, n-1$ 都有 a_j 等于 `a[j] + add[j / B]`。

为了回答 $a_l, \ldots, a_r$ 中小于 c^2 的数的个数，

- 每个块 i，维护块内元素对应的那些 `a[j]` 的**有序**序列 `sorted[i]`。

区间加

- 对于整块 i，更新 `add[i]`。
- 对于边块 i，首先更新其中被修改的元素 a_j 对应的 `a[j]`，然后**重建**块 i 的有序列表 `sorted[i]`（重新排序）。

时间：$O(\sqrt{n} + \sqrt{n}\log\sqrt{n}) = O(\sqrt{n}\log n)$

区间查询

- 对于整块 i，在 `sorted[i]` 里做二分查找。
- 对于边块，逐个检查涉及的元素。

时间：$O(\sqrt{n} + \sqrt{n}\log\sqrt{n}) = O(\sqrt{n}\log n)$

```cpp
int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    int n; cin >> n;
    vector<long long> a(n);
    for (int i = 0; i < n; i++)
        cin >> a[i];

    int B = (int) sqrt(n);
    int NB = (n + B - 1) / B;

    vector<vector<long long>> b(NB); // b 即 sorted
    vector<long long> add(NB);

    auto build = [&](int i) {
        int l = i * B, r = min(l + B, n);
        b[i].assign(a.begin() + l, a.begin() + r);
        sort(b[i].begin(), b[i].end());
    };

    for (int i = 0; i < NB; i++)
        build(i);
```

```cpp
    int q = n;
    while (q--) {
        int type, l, r; long long c;
        cin >> type >> l >> r >> c;
        l--;
        int lb = (l + B - 1) / B;
        int rb = r / B;
        if (type == 0) {
            if (lb > rb) {
                for (int i = l; i < r; i++)
                    a[i] += c;
                build(lb - 1);
            } else {
                for (int i = l; i < lb * B; i++)
                    a[i] += c;
                for (int i = rb * B; i < r; i++)
                    a[i] += c;
                for (int i = lb; i < rb; i++)
                    add[i] += c;
                if (l != lb * B)
                    build(lb - 1);
                if (r != rb * B)
                    build(rb);
            }
        } else {
            c *= c;
            int ans = 0;
            if (lb > rb) {
                for (int i = l; i < r; i++)
                    ans += (a[i] + add[lb - 1]) < c;
            } else {
                for (int i = l; i < lb * B; i++)
                    ans += (a[i] + add[lb - 1]) < c;
                for (int i = rb * B; i < r; i++)
                    ans += (a[i] + add[rb]) < c;
                for (int i = lb; i < rb; i++)
                    ans += lower_bound(b[i].begin(), b[i].end(), c - add[i]) - b[i].begin();
            }
            cout << ans << '\n';
        }
    }
}
```

一个优化

在处理区间加时，把边块重新排序，可以通过**归并**来实现。

对于边块来说，被加上 c 的是其中的一段元素。修改之后，那一段仍是有序的。边块里没被修改的那些元素也是有序的。

数列分块入门 3

给你一个长为 n 的整数序列 $a = (a_1, \ldots, a_n)$。处理 n 个操作。操作有两类：

- `0 l r c`：把 $a_l, \ldots, a_r$ 每个都加上 c。
- `1 l r c`：询问 $a_l, \ldots, a_r$ 中小于 c 的最大的数。若不存在小于 c 的数，输出 -1。

限制

- $1 \le n \le 200000$
- a_i 的初始值，c，每次操作后的 a_i 都在 int 范围内。

解法与《数列分块入门 2》类似。

```
int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    int n; cin >> n;
    vector<long long> a(n);
    for (int i = 0; i < n; i++)
        cin >> a[i];

    int B = (int) sqrt(n);
    int NB = (n + B - 1) / B;

    vector<vector<long long>> b(NB);
    vector<long long> add(NB);

    auto build = [&](int i) {
        int l = i * B, r = min(l + B, n);
        b[i].assign(a.begin() + l, a.begin() + r);
        sort(b[i].begin(), b[i].end());
    };

    for (int i = 0; i < NB; i++)
        build(i);
    int q = n;
    while (q--) {
        int type, l, r, c;
        cin >> type >> l >> r >> c;
        l--;
        int lb = (l + B - 1) / B;
        int rb = r / B;
```

```
        if (type == 0) {
            if (lb > rb) {
                for (int i = l; i < r; i++)
                    a[i] += c;
                build(lb - 1);
            } else {
                for (int i = l; i < lb * B; i++)
                    a[i] += c;
                for (int i = rb * B; i < r; i++)
                    a[i] += c;
                for (int i = lb; i < rb; i++)
                    add[i] += c;
                if (l != lb * B)
                    build(lb - 1);
                if (r != rb * B)
                    build(rb);
            }
        } else {
            long long ans = LLONG_MIN;
            if (lb > rb) {
                for (int i = l; i < r; i++)
                    if (a[i] + add[lb - 1] < c)
                        ans = max(ans, a[i] + add[lb - 1]);
            } else {
                for (int i = l; i < lb * B; i++)
                    if (a[i] + add[lb - 1] < c)
                        ans = max(ans, a[i] + add[lb - 1]);
                for (int i = rb * B; i < r; i++) {
                    if (a[i] + add[rb] < c)
                        ans = max(ans, a[i] + add[rb]);
                }
                for (int i = lb; i < rb; i++) {
                    auto it = lower_bound(b[i].begin(), b[i].end(), c - add[i]);
                    if (it != b[i].begin())
                        ans = max(ans, *prev(it) + add[i]);
                }
            }
            if (ans == LLONG_MIN)
                ans = -1;
            cout << ans << '\n';
        }
    }
}
```

数列分块入门 4

给你一个长为 n 的整数序列 $a = (a_1, \ldots, a_n)$。处理 n 个操作，操作有两类：

- `0 l r c`：把 $a_l, \ldots, a_r$ 每个都加上 c。
- `1 l r c`：输出 $(a_l + \cdots + a_r)$ 除以 $(c + 1)$ 的余数。保证 $c \geq 0$。

限制

- $1 \leq n \leq 3 \times 10^5$
- a_i 的初始值，c，每次操作后的 a_i 都在 int 范围内。

分块

取 $B = \lfloor \sqrt{n} \rfloor$。

- 对每个块 i，维护对块 i 的整体加的操作所加的数的总和 `add[i]`。
- 维护一个长为 n 的数组 `a`。
  对于每个 $j = 0, \ldots, n-1$ 都有 a_j 等于 `a[j] + add[j / B]`。
- 对每个块 i，维护块内元素对应的那些 `a[j]`，维护 `sum[i]`。

区间加

- 对于整块 i，更新 `add[i]`。
- 对于边块 i，更新其中被修改的元素 a_j 对应的 `a[j]`，更新 `sum[i]`。

时间：$O(\sqrt{n})$

查询区间和

- 对于整块 i，其中元素的和等于 `sum[i] + add[i] * B`。
- 对于边块，把涉及的元素逐个加起来。

时间：$O(\sqrt{n})$

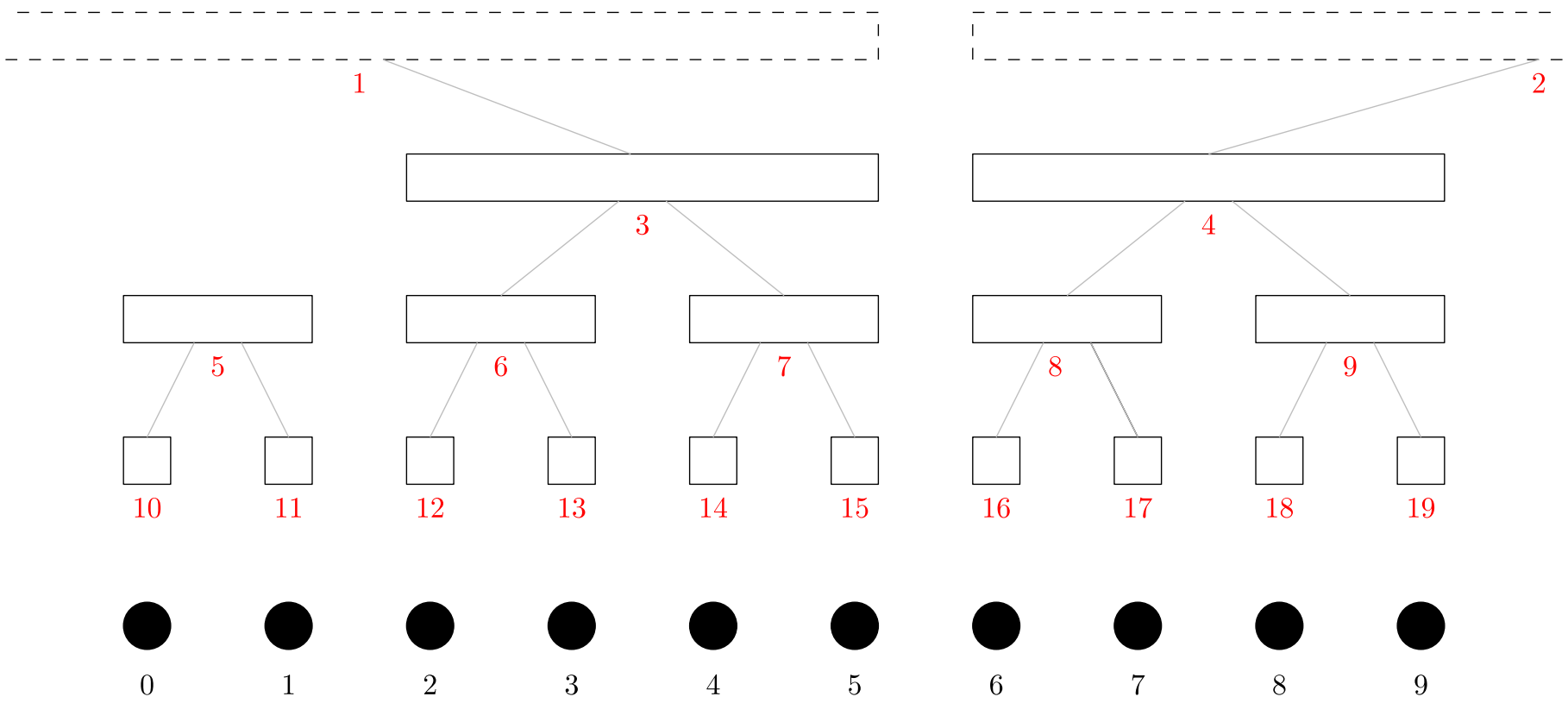
```cpp
int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    int n;
    cin >> n;
    vector<long long> a(n);
    for (int i = 0; i < n; i++)
        cin >> a[i];

    int B = (int) sqrt(n);
    int NB = (n + B - 1) / B;

    vector<long long> add(NB);
    vector<long long> sum(NB);
    for (int i = 0; i < n; i++)
        sum[i / B] += a[i];
```

```cpp
    int q = n;
    while (q--) {
        int type, l, r, c;
        cin >> type >> l >> r >> c;
        l--;
        int lb = (l + B - 1) / B;
        int rb = r / B;
        if (type == 0) {
            if (lb > rb) {
                for (int i = l; i < r; i++) {
                    a[i] += c;
                    sum[lb - 1] += c;
                }
            } else {
                for (int i = l; i < lb * B; i++) {
                    a[i] += c;
                    sum[lb - 1] += c;
                }
                for (int i = rb * B; i < r; i++) {
                    a[i] += c;
                    sum[rb] += c;
                }
                for (int i = lb; i < rb; i++)
                    add[i] += c;
            }
        } else {
            long long ans = 0;
            if (lb > rb)
                for (int i = l; i < r; i++)
                    ans += a[i] + add[lb - 1];
            else {
                for (int i = l; i < lb * B; i++)
                    ans += a[i] + add[lb - 1];
                for (int i = rb * B; i < r; i++)
                    ans += a[i] + add[rb];
                for (int i = lb; i < rb; i++)
                    ans += sum[i] + add[i] * B;
            }
            ans %= c + 1;
            if (ans < 0) ans += c + 1;
            cout << ans << '\n';
        }
    }
}
```

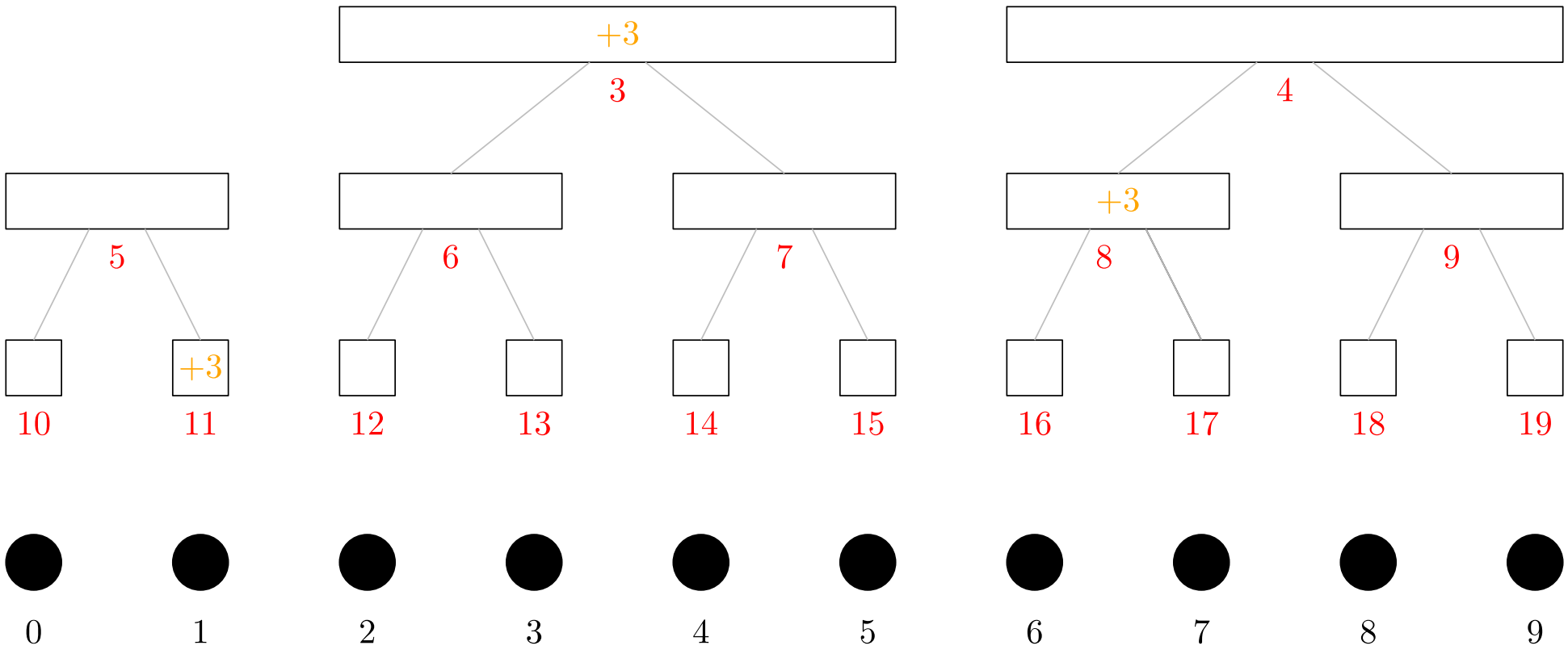
用线段树处理「区间加，查询区间和」



对于每个块 i，维护两个数据

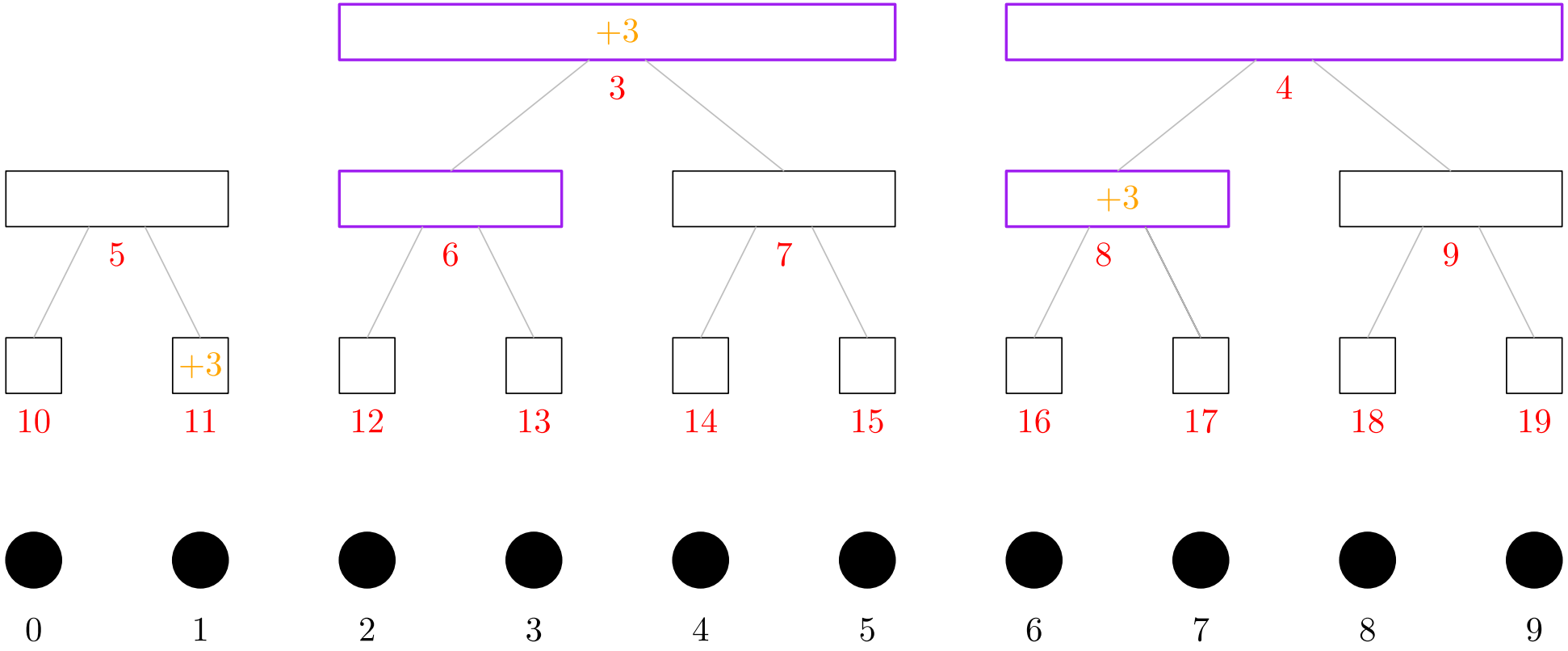
- **标记**：块 i 被整体加的数的总和 `add[i]`。
- **值**：块 i 里的数之和 `sum[i]`。

给 a_1 到 a_7 加 3



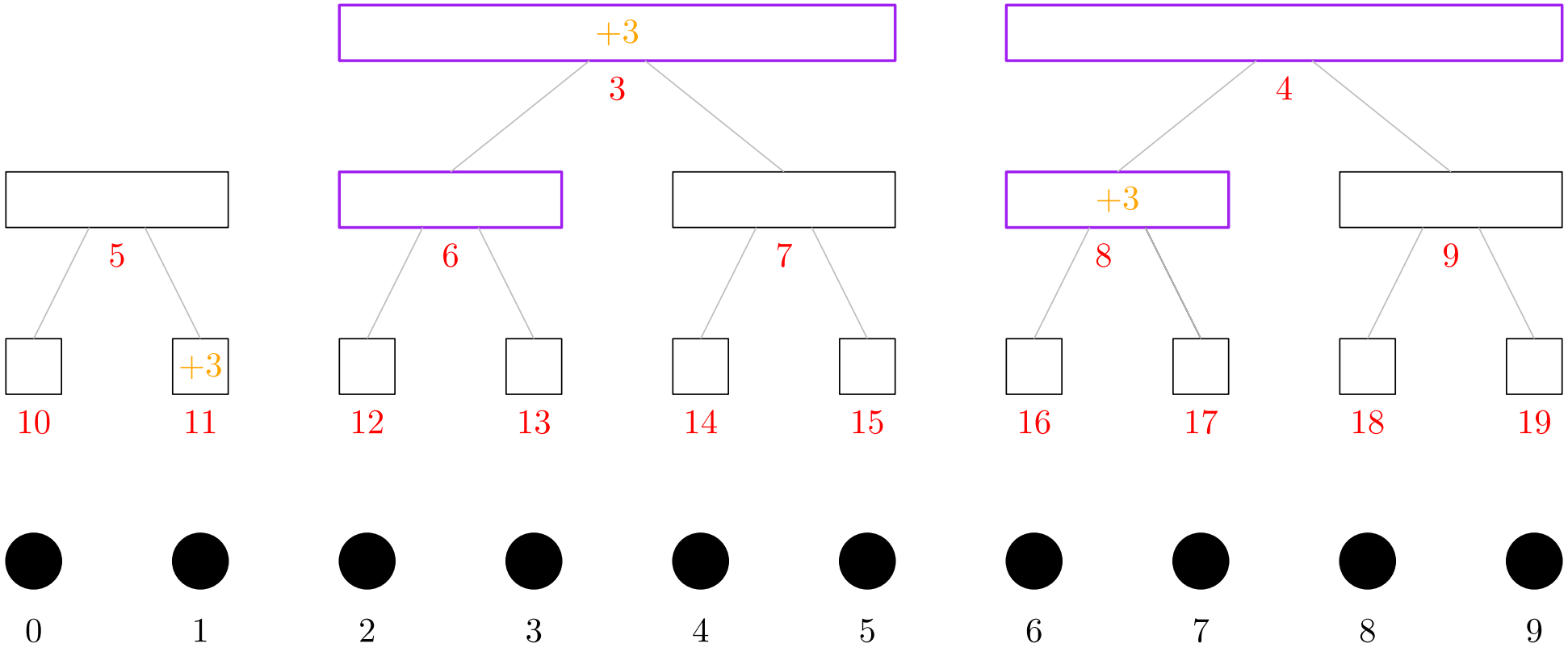
1. 按从上到下的顺序，下传边块（5 和 4）的标记 `add[j]`。
2. 更新极大整块（11、8、3）的标记和值。
3. 按从下到上的顺序，更新边块的值。

查询 a_3 到 a_6 的和



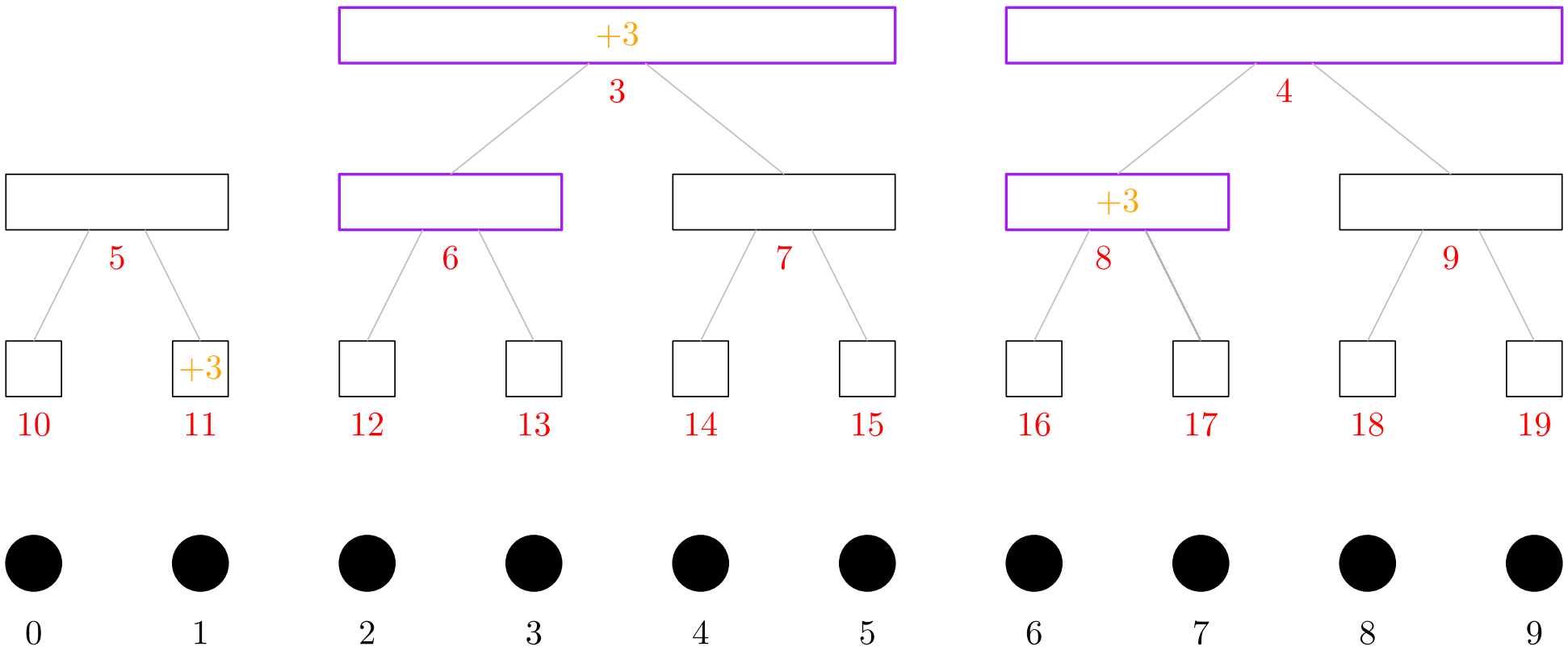
1. 按从上到下的顺序，下传边块（3、4、6，8）的标记。
2. 查询极大整块（13、16、7）的值。

找边块

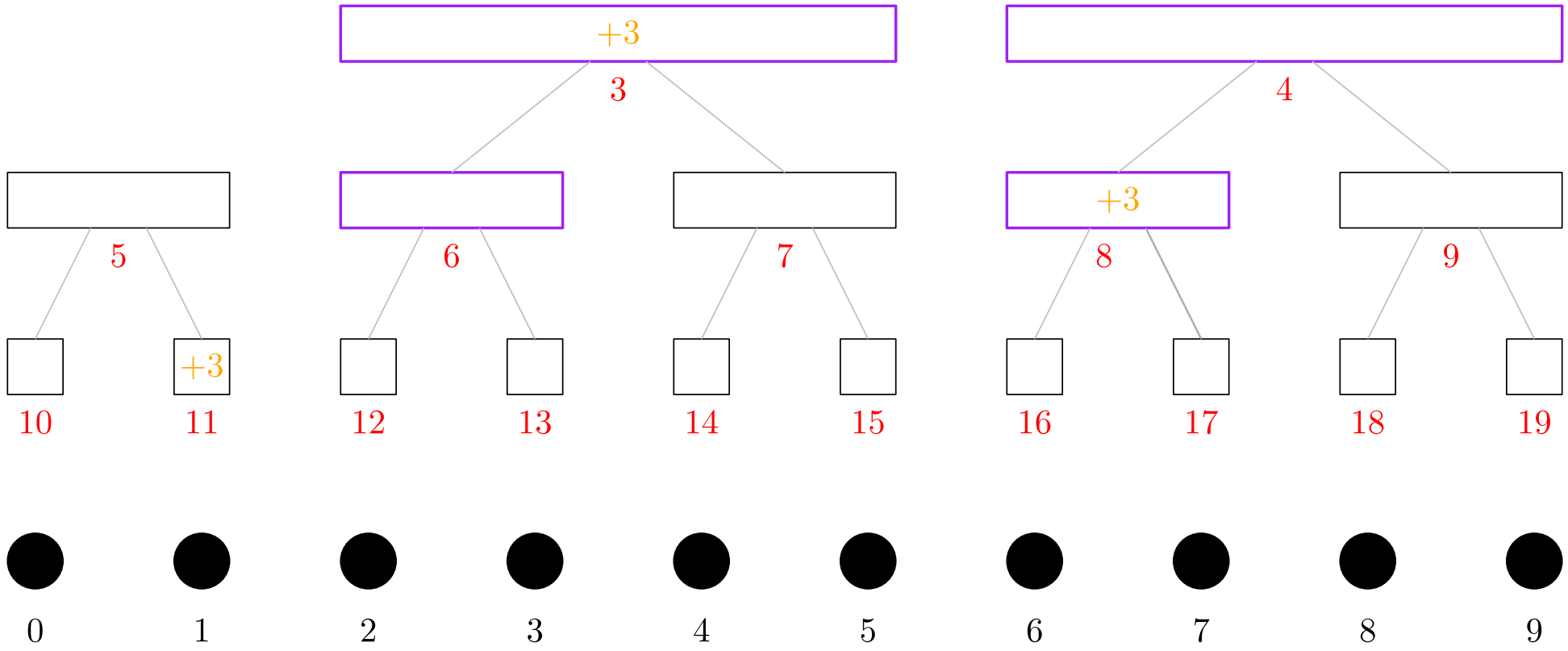


对于区间 $[3, 7)$ 来说，边块是

- 13 的那些**不全在自己右边**的祖先，即 6 和 3。
- 17 的那些**不全在自己右边**的祖先，即 8 和 4。



节点（也就是块）i 的某个祖先 j 不全在 i 右边。也就是说从节点 i 走到祖先 j **不全是往右上走的**。也就是从说从 i 到 j，节点编号不是一直除以 2 的。

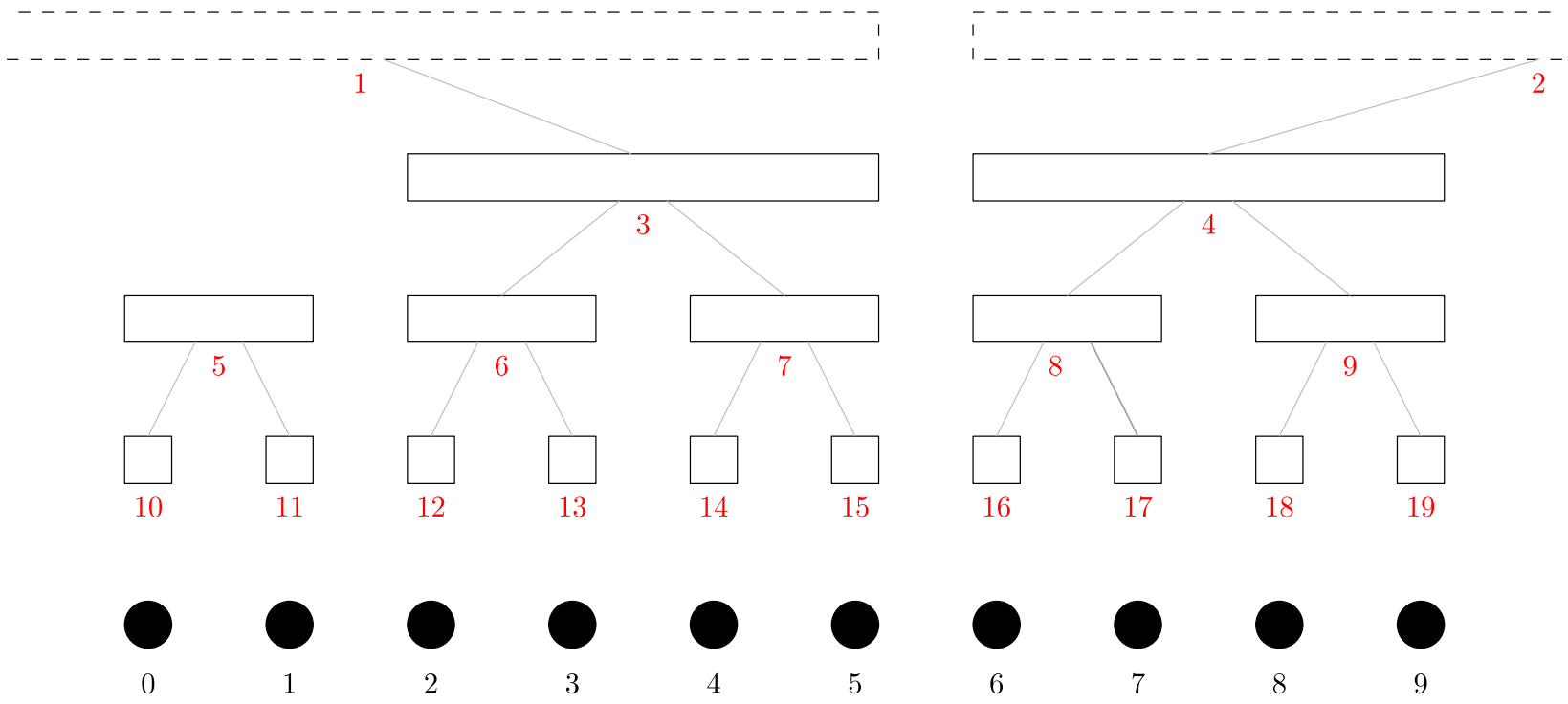


设 j 是 i 的第 k 个祖先，那么 j 等于 `i >> k`。

从 i 走到祖先 j 不全是往右上走的，

- 也就是说 i 的二进制写法的末尾没有 k 个连续的零。
- 也就是说 `i >> k << k != i`。

线段树有多少层



- 长为 n 的序列的线段树有 $\lfloor \log_2 n \rfloor + 1$ 层。如果不算最后一层，有 $\lfloor \log_2 n \rfloor$ 层。
- 最上面那一层可能用不到。
- $\lfloor \log_2 n \rfloor + 1$ 是正整数 n 的二进制写法的位数。计算方法
  - `32 - __builtin_clz(n)` clz：count leading zero
  - `bit_width((unsigned) n)` since C++20

用线段树处理「区间加，查询区间和」

```cpp
int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    int n; cin >> n;

    vector<long long> add(2 * n), sum(2 * n);

    for (int i = 0; i < n; i++)
        cin >> sum[i + n];
    for (int i = n - 1; i > 0; i--)
        sum[i] = sum[2 * i] + sum[2 * i + 1];

    int LOG = bit_width((unsigned) n);

    auto apply = [&](int p, long long c, int len) {
        add[p] += c;
        sum[p] += c * len;
    };

    auto update = [&](int p) {
        sum[p] = sum[p * 2] + sum[p * 2 + 1];
    };

    auto push = [&](int p, int len) {
        apply(p * 2, add[p], len / 2);
        apply(p * 2 + 1, add[p], len / 2);
        add[p] = 0;
    };
```

```cpp
    int q = n;
    while (q--) {
        int type, l, r, c;
        cin >> type >> l >> r >> c;
        l--;
        l += n; r += n;
        if (type == 0) {
            for (int i = LOG - 1; i >= 1; i--) {
                if (l >> i << i != l) push(l >> i, 1 << i);
                if (r >> i << i != r) push(r >> i, 1 << i);
            }
            {
                int l2 = l, r2 = r;
                int len = 1;
                while (l < r) {
                    if (l & 1) apply(l++, c, len);
                    if (r & 1) apply(--r, c, len);
                    l >>= 1; r >>= 1;
                    len *= 2;
                }
                l = l2; r = r2;
            }
            for (int i = 1; i < LOG; i++) {
                if (l >> i << i != l) update(l >> i);
                if (r >> i << i != r) update(r >> i);
            }
        } else {
            for (int i = LOG - 1; i >= 1; i--) {
                if (l >> i << i != l) push(l >> i, 1 << i);
                if (r >> i << i != r) push(r >> i, 1 << i);
            }
            long long ans = 0;
            while (l < r) {
                if (l & 1) ans += sum[l++];
                if (r & 1) ans += sum[--r];
                l >>= 1; r >>= 1;
            }
            ans %= c + 1;
            if (ans < 0) ans += c + 1;
            cout << ans << '\n';
        }
    }
}
```

数列分块入门 5

给你一个长为 n 的整数序列 $a = (a_1, \ldots, a_n)$。处理 n 个操作，操作有两类：

- `0 l r c`：把 $a_l, \ldots, a_r$ 每个都开平方。也就是把 a_i 变成 $\lfloor \sqrt{a_i} \rfloor$（$l \le i \le r$）。
- `1 l r c`：输出 $(a_l + \cdots + a_r)$。

限制

- $1 \le n \le 3 \times 10^5$
- $0 \le a_i \le 2^{31} - 1$

对 $2^{31}-1$ 开平方，开 5 次平方之后就会得到 1。

```
int x = INT_MAX;
while (x > 1) {
    x = sqrt(x);
    cout << x << '\n';
}
```

输出

```
46340
215
14
3
1
```

分块

取 $B = \lfloor \sqrt{n} \rfloor$。

- 对每个块 i，
  - 维护一个列表 `big[i]`，是这一块中大于 1 的元素的下标。
  - 维护其中的元素之和 `sum[i]`。
- 维护一个长为 n 的数组 `a`，即序列 a。

区间开平方

- 对于整块 i，枚举列表 `big[i]` 中的下标 j，把 a_j 开平方。若 a_j 变成 1，则把 j 从 `big[i]` 中删除。
- 对于边块 i，把涉及的元素 a_j 开平方。即使 a_j 从大于 1 变成 1，也不把 j 从 `big[i]` 中删除。

一个元素在整块里被开平方至多 5 次。

查询区间和

- 对于整块 i，其中元素的和等于 `sum[i]`。
- 对于边块，把涉及的元素逐个加起来。

时间：$O(\sqrt{n})$

```cpp
int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    int n;
    cin >> n;
    vector<long long> a(n);
    for (int i = 0; i < n; i++)
        cin >> a[i];

    int B = (int) sqrt(n);
    int NB = (n + B - 1) / B;

    vector<vector<int>> big(NB);
    vector<long long> sum(NB);
    for (int i = 0; i < n; i++) {
        sum[i / B] += a[i];
        if (a[i] > 1)
            big[i / B].push_back(i);
    }

    auto update = [&](int i) {
        int x = (int) sqrt(a[i]);
        sum[i / B] += x - a[i];
        a[i] = x;
    };
```

```cpp
    int q = n;
    while (q--) {
        int type, l, r, c;
        cin >> type >> l >> r >> c;
        if (l > r) swap(l, r);
        l--;
        int lb = (l + B - 1) / B;
        int rb = r / B;
        if (type == 0) {
            if (lb > rb)
                for (int i = l; i < r; i++) update(i);
            else {
                for (int i = l; i < lb * B; i++) update(i);
                for (int i = rb * B; i < r; i++) update(i);
                for (int i = lb; i < rb; i++)
                    for (int j = 0; j < (int) big[i].size(); ) {
                        update(big[i][j]);
                        if (a[big[i][j]] == 1) {
                            swap(big[i][j], big[i].back());
                            big[i].pop_back();
                        } else {
                            j++;
                        }
                    }
            }
        } else {
            long long ans = 0;
            if (lb > rb)
                for (int i = l; i < r; i++) ans += a[i];
            else {
                for (int i = l; i < lb * B; i++) ans += a[i];
                for (int i = rb * B; i < r; i++) ans += a[i];
                for (int i = lb; i < rb; i++) ans += sum[i];
            }
            cout << ans << '\n';
        }
    }
}
```

数列分块入门 6

给你一个长为 n 的整数序列 $a = (a_1, \ldots, a_n)$。处理 n 个操作，操作有两类：

- `0 l r`：在 a_l 之前插入 r。
- `1 c`：询问 a_c 的值。

限制

- $1 \le n \le 3 \times 10^5$
- a_i，r 在 int 范围内。
- $1 \le l, c \le$ 操作时序列 a 的长度。
- 测试数据随机生成：先随机等概率地生成类型（0 或 1），其余询问参数在所有合法的值中随机等概率地抽取。

序列 a 的最终长度的期望值是 $1.5 \cdot n$。

我们取块长 $B = \lfloor \sqrt{1.5 \cdot n}/2 \rfloor$，对初始的序列 a 分块。
把每一块存储在一个 `vector<int>` 里。

在位置 p 插入元素 x

1. 找到 x 应该插入到哪一块。
2. 把 x 插入到那一块，用 `vector::insert()` 。
3. 如果那一块的长度达到 $2B$，就把它分成两个长为 B 的块。

这个数据结构叫作**块状链表**。

```cpp
int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    int n; cin >> n;

    int B = 333;
    int NB = (n + B - 1) / B;

    vector<vector<int>> b(NB);

    for (int i = 0; i < n; i++) {
        int x; cin >> x;
        b[i / B].push_back(x);
    }
    int q = n;
```

```cpp
    while (q--) {
        int type; cin >> type;
        if (type == 0) {
            int p, x; cin >> p >> x;
            p--;
            for (int i = 0; i < (int) b.size(); i++) {
                if (p <= (int) b[i].size()) {
                    b[i].insert(b[i].begin() + p, x);
                    if ((int) b[i].size() == 2 * B) {
                        vector<int> t(b[i].begin() + B, b[i].end());
                        b.insert(b.begin() + i + 1, t);
                        b[i].resize(B);
                    }
                    break;
                }
                p -= (int) b[i].size();
            }
        } else {
            int p; cin >> p;
            p--;
            for (int i = 0; i < (int) b.size(); i++) {
                if (p < (int) b[i].size()) {
                    cout << b[i][p] << '\n';
                    break;
                }
                p -= (int) b[i].size();
            }
        }
    }
}
```

数列分块入门 7

给你一个长为 n 的整数序列 $a = (a_1, \ldots, a_n)$。处理 n 个操作，操作有三类：

- `0 l r c`：把 $a_l, \ldots, a_r$ 每个都加 c。
- `1 l r c`：把 $a_l, \ldots, a_r$ 每个都乘 c。
- `1 l r c`：询问 a_r 的值模 10007（忽略 l 和 c）。

限制

- $1 \le n \le 3 \times 10^5$
- a_i，c 在 int 范围内。

分块

取 $B = \lfloor \sqrt{n} \rfloor$。

对每个块 i，我们维护

- **标记**：对它的整体修改操作 `tag[i]`。

`tag[i]` = (x, y)，表示对块 i 里的每个数先乘以 x，再加上 y。

整体加 c 可表示为 $(1, c)$，整体乘 c 可表示为 $(c, 0)$。

先做 (x_1, y_1) 再做 (x_2, y_2)，合起来可表示为 $(x_2 x_1, x_2 y_1 + y_2)$。
这叫作操作的**复合**或**合成**。

另外我们维护一个数组 `a`，表示序列 a，但不需要实时更新。

区间修改

对于整块 i，更新 `tag[i]`。

对于边块 i，把 `tag[i]` 下传到数组 `a`，然后逐个修改当前块内涉及的元素。

单点查询

把 `tag[i / B]` 作用到 `a[i]`，就得到当前的 a_i。

```cpp
int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    const int mod = 10007;
    int n;
    cin >> n;
    vector<int> a(n);
    for (int i = 0; i < n; i++) {
        cin >> a[i];
        a[i] %= mod;
        if (a[i] < 0) a[i] += mod;
    }
    struct F {
        int mul = 1;
        int add = 0;
    };

    int B = (int) sqrt(n);
    int NB = (n + B - 1) / B;
    vector<F> lz(NB); // lz 是 lazy 的简写


    auto push = [&](int ib) {
        for (int i = ib * B; i < min(n, (ib + 1) * B); i++)
            a[i] = (a[i] * lz[ib].mul + lz[ib].add) % mod;
        lz[ib] = {1, 0};
    };

    int q = n;
```

```cpp
    while (q--) {
        int type, l, r, c;
        cin >> type >> l >> r >> c;
        if (type == 2) {
            r--;
            int i = r / B;
            cout << (a[r] * lz[i].mul + lz[i].add) % mod << '\n';
        } else {
            l--;
            int lb = (l + B - 1) / B;
            int rb = r / B;
            c %= mod;
            if (c < 0) c += mod;
            F f;
            if (type == 0) f = {1, c};
            else f = {c, 0};
            if (lb > rb) {
                push(lb - 1);
                for (int i = l; i < r; i++) {
                    a[i] = (a[i] * f.mul + f.add) % mod;
                }
            } else {
                if (l < lb * B) {
                    push(lb - 1);
                    for (int i = l; i < lb * B; i++)
                        a[i] = (a[i] * f.mul + f.add) % mod;
                }
                if (rb * B < r) {
                    push(rb);
                    for (int i = rb * B; i < r; i++)
                        a[i] = (a[i] * f.mul + f.add) % mod;
                }
                for (int i = lb; i < rb; i++) {
                    lz[i].mul = (lz[i].mul * f.mul) % mod;
                    lz[i].add = (lz[i].add * f.mul + f.add) % mod;
                }
            }
        }
    }
}
```

用线段树处理「区间修改，单点查询」

《数列分块入门 1》是「区间加，单点查询」，这题的修改操作和区间加的区别在于：

- 加是不讲顺序的而先乘再加是讲顺序的。

例如，“先加 3 再乘 2” 和“先乘 2 再加 3”是不一样的。

> $(1,3)$ 与 $(2,0)$ 复合的结果是 $(2,6)$，而 $(2,0)$ 与 $(1,3)$ 复合的结果是 $(2,3)$。

我们把不讲顺序的操作称为**可交换的**，讲顺序的操作称为**不可交换的**。

区间修改

1. 按从上往下的顺序下传边块的标记。
2. 把当前操作作用在极大整块上：更新它的标记。

单点查询

按**从下到上**的顺序把 a_i 所属的块的标记作用在 a_i 的初始值上，结果就是 a_i 的当前值。

```cpp
struct F {
    int mul = 1;
    int add = 0;
};

F composite(F x, F y) { // 先 y 后 x
    return {x.mul * y.mul % mod, (x.mul * y.add + x.add) % mod};
};

int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);

    int n; cin >> n;
    vector<int> a(n);
    for (int i = 0; i < n; i++) {
        cin >> a[i];
        a[i] %= mod;
    }

    vector<F> lz(2 * n);

    auto apply = [&](int i, F f) {
        lz[i] = composite(f, lz[i]);
    };
    auto push = [&](int i) { // 下传标记
        apply(2 * i, lz[i]);
        apply(2 * i + 1, lz[i]);
        lz[i] = {1, 0};
    };

    int LOG = 31 - __builtin_clz(n); // log2(n) 向下取整
    int q = n;
```

```cpp
    while (q--) {
        int type, l, r, c;
        cin >> type >> l >> r >> c;
        if (type == 2) {
            int v = a[r - 1];
            int p = r - 1 + n;
            while (p > 0) {
                v = (lz[p].mul * v + lz[p].add) % mod;
                p /= 2;
            }
            if (v < 0) v += mod;
            cout << v << '\n';
        } else {
            c %= mod;
            F f;
            if (type == 0) f = {1, c};
            else f = {c, 0};
            l--;
            l += n; r += n;
            for (int i = LOG; i >= 1; i--) {
                if (l >> i << i != l)
                    push(l >> i);
                if (r >> i << i != r)
                    push(r >> i);
            }
            while (l < r) {
                if (l & 1) apply(l++, f);
                if (r & 1) apply(--r, f);
                l /= 2; r /= 2;
            }
        }
    }
}
```

数列分块入门 8

给你一个长为 n 的整数序列 $a = (a_1, \ldots, a_n)$。处理 n 个操作

- `l r c`：先输出 $a_l, \ldots, a_r$ 中有多少个 c，然后把 $a_l, \ldots, a_r$ 都改为 c。

限制

- $1 \le n \le 3 \times 10^5$
- a_i，c 在 int 范围内。

分块

取 $B = \lfloor \sqrt{n} \rfloor$。

对每个块 i，维护两个值

- `mark[i]`：bool 值，块 i 最近一次经历的整体赋值有没有下传。
- `val[i]`：当 `mark[i]` 为 `true` 时，它表示块 i 被整体赋的值。

区间操作

- 对于边块 i，先下传标记，然后逐个检查、修改涉及的项。
- 对于整块 i，如果它有标记，看 `val[i]` 是否等于 c，然后把 `val[i]` 改为 c；如果它没有标记，逐个检查每一项，然后打上标记。

每一个块在第一次作整块之后就有了标记，此后它失去标记当且仅当它作边块。
所有块作边块的总次数不超过 $2q$，所以

- 所有块作为没有标记的整块的总次数不超过 $2q+$ 总块数。

```cpp
int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    int n;
    cin >> n;
    vector<int> a(n);
    for (int i = 0; i < n; i++)
        cin >> a[i];

    int B = (int) sqrt(n);
    int NB = (n + B - 1) / B;

    vector<bool> mark(NB);
    vector<int> val(NB);

    auto push = [&](int ib) {
        if (mark[ib]) {
            for (int i = ib * B; i < (ib + 1) * B; i++) {
                a[i] = val[ib];
            }
            mark[ib] = false;
        }
    };

    int q = n;
```

```cpp
    while (q--) {
        int l, r, c;
        cin >> l >> r >> c;
        l--;
        int lb = (l + B - 1) / B;
        int rb = r / B;
        int ans = 0;
        if (lb > rb) {
            push(lb - 1);
            for (int i = l; i < r; i++) {
                ans += a[i] == c;
                a[i] = c;
            }
        } else {
            if (l < lb * B) {
                push(lb - 1);
                for (int i = l; i < lb * B; i++) {
                    ans += a[i] == c;
                    a[i] = c;
                }
            }
            if (rb * B < r) {
                push(rb);
                for (int i = rb * B; i < r; i++) {
                    ans += a[i] == c;
                    a[i] = c;
                }
            }
            for (int i = lb; i < rb; i++) {
                if (mark[i]) {
                    ans += (val[i] == c) * B;
                } else {
                    for (int j = i * B; j < (i + 1) * B; j++)
                        ans += a[j] == c;
                }
                mark[i] = true;
                val[i] = c;
            }
        }
        cout << ans << '\n';
    }
}
```

区间推平

「把 $a_l, \ldots, a_r$ 都改为 c」这样的操作，也被称为**区间推平**。

处理区间推平操作的另一个办法是**维护同值的连续段**：

- 用一个 `map<下标类型，值类型>` 来表示一个序列，对于每个同值的连续段，只存这一段的第一项的下标和值。
- 例如，序列 $a = (1, 2, 2, 3, 9, 2)$ 被表示为 $\{(0, 1), (1, 2), (3, 3), (4, 9), (5, 2)\}$（下标从零开始）。我们把下标 $0, 1, 3, 4, 5$ 称为**端点**。

这种表示序列的方法，也可看作一种分块：把每个同值的连续段作为一块。

一次区间推平操作的影响：边块缩短，整块消失，新增一个块。

```cpp
int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    int n;
    cin >> n;
    map<int,int> a;
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }
    int q = n;
    while (q--) {
        int l, r, c;
        cin >> l >> r >> c;
        --l;
        auto lptr = a.insert({l, prev(a.upper_bound(l))->second}).first;
        auto rptr = a.insert({r, prev(a.upper_bound(r))->second}).first;
        int ans = 0;
        for (auto it = lptr; it != rptr; ++it) {
            auto [i, v] = *it;
            if (v == c) {
                ans += next(it)->first - i;
            }
        }
        a.erase(lptr, rptr);
        a[l] = c;
        cout << ans << '\n';
    }
}
```

推平区间 $[l, r)$

1. 查询 a_l, a_r 的值，把 l, r 设置为端点。
2. 查询区间 $[l, r)$。
3. 删除区间 $[l, r)$ 上的端点。
4. `a[l] = c;` 。
   （把 l 设置为端点，并把 a_l 置为 c。也可以写 `a.insert({l, c});` 。）

```
auto lptr = a.insert({l, prev(a.upper_bound(l))->second}).first;
```

- `prev(a.upper_bound(l))->second` 是 a_l 的值。
- `a.insert({l, al})` 尝试把 `{l, al}` 插入 map。
  返回一个 `pair<iterator, bool>`，.second 表示插入是否成功，.first 表示指向 `l` 对应的 pair 的迭代器（相当于指针）。

Points

Codeforces 19D

给你一个平面直角坐标系。处理 n 个操作，操作有三种

- `add x y`：给点 (x, y) 打上标记。保证此时点 (x, y) 上没有标记。
- `remove x y`：擦除点 (x, y) 上的标记。保证此时点 (x, y) 上有标记。
- `find x y`：检查是否有被标记的点 (x', y') 满足 $x' > x$ 且 $y' > y$。若有，输出一个这样的点的坐标。若有多个，输出字典序最小的坐标。若没有，输出 -1。

$1 \le n \le 2 \cdot 10^5$，点的坐标是不超过 10^9 的非负整数。

离线处理操作

- 读取 n 个操作。
- 对读入的 n 个点的横坐标进行**压缩**。例：$(30, 11, 42, 30) \to (1, 0, 2, 1)$。设压缩过后点的横坐标的范围是 $[0, N)$。

分块

- 取 $B = \lfloor \sqrt{N} \rfloor$，对横坐标的取值范围进行分块。这种分块我们称之为**值域分块**。
- 对每个块 i，用一个 mutiset<int> `b[i]` 存储横坐标在块 i 里的那些有标记的点的纵坐标。
- 对每个横坐标 j，用一个 set<int> `a[j]` 存储横坐标等于 j 的那些有标记的点的纵坐标。

区间查询

对于询问 `find x y`，我们要找出 $[x+1, N)$ 范围内第一个满足下述条件的横坐标 j

- `a[j]` 里的最大元素大于 y 的横坐标 j。

为了快速找出这个横坐标，

- 对于边块，逐个枚举其中相关的横坐标 j，检查 `a[j]` 的最大元素是否大于 y。
- 对于整块 i，如果 `b[i]` 的最大值大于 y，就逐个检查其中的横坐标。

当我们得到所求的横坐标 j 后，再查找 `a[j]` 里第一个大于 y 的值即可。

单点修改

对于操作 `add x y` 或 `move x y`，修改 `a[x]` 和 `b[x / B]`。

坐标压缩

```
vector<int> compress(vector<int>& a) {
    vector<pair<int,int>> vi;
    for (int i = 0; i < (int) a.size(); i++)
        vi.push_back({a[i], i});

    sort(vi.begin(), vi.end());

    vector<int> b;
    for (auto [v, i] : vi) {
        if (b.empty() || b.back() < v)
            b.push_back(a[i]);
        a[i] = (int) b.size() - 1;
    }
    return b; // 排序去重之后的 a
}
```

```cpp
int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    int n;
    cin >> n;
    vector<string> type(n);
    vector<int> x(n), y(n);
    for (int i = 0; i < n; i++)
        cin >> type[i] >> x[i] >> y[i];

    vector<int> real_x = compress(x);

    int N = (int) real_x.size();
    int B = (int) sqrt(N);
    int NB = (N + B - 1) / B;

    vector<set<int>> a(N);
    vector<multiset<int>> b(NB);
```

```cpp
auto work = [&](int l, int r, auto ok) -> int {
   int lb = (l + B - 1) / B;
   int rb = r / B;
   if (lb > rb) {
       for (int i = l; i < r; i++)
           if (ok(a[i]))
               return i;
       return r;
   }

   for (int i = l; i < lb * B; i++)
       if (ok(a[i]))
           return i;

   for (int ib = lb; ib < rb; ib++)
       if (ok(b[ib]))
           for (int i = ib * B; i < (ib + 1) * B; i++)
               if (ok(a[i]))
                   return i;

   for (int i = rb * B; i < r; i++)
       if (ok(a[i]))
           return i;

   return r;
};
```

```
    for (int i = 0; i < n; i++) {
        if (type[i] == "add") {
            a[x[i]].insert(y[i]);
            b[x[i] / B].insert(y[i]);
        } else if (type[i] == "remove") {
            a[x[i]].e(y[i]); // 删除一个y[i]，since C++17
            b[x[i] / B].extract(y[i]);
        } else {
            auto ok = [y = y[i]](const multiset<int>& s) {
                return s.size() && *s.rbegin() > y;
            };
            int j = work(x[i] + 1, N, ok);
            if (j == N)
                cout << -1 << '\n';
            else
                cout << real_x[j] << ' ' << *a[j].upper_bound(y[i]) << '\n';
        }
    }
}
```

用线段树解决这题

```
vector<int> b(2 * N, -1);
vector<set<int>> a(N);
```

线段树的每个节点 `b[i]` 维护「横坐标在相应范围内的点」的纵坐标的最大值。

每对个横坐标 x，用一个 set<int> `a[x]` 来存储横坐标等于 x 的点的纵坐标。

单点修改

对于操作 `add x y` 或 `remove x y`，

1. 修改 `a[x]`
2. 更新线段树的叶子 `b[x + N]`
3. 从下往上，更新 `x + N` 的祖先节点

区间查询

`find x, y`：找出区间 $[x+1, N)$ 中第一个满足 `b[i + N] > y` 的 `i`。

1. 从下往上，把所查询的区间 $[x+1, N)$ 拆解为极大整块，找出从左到右第一个满足 `b[k] > y` 的极大整块 k。

   在每一层

   - 如果左边的极大整块 l 满足 `b[l] > y`，那么 l 就是 k，结束。
   - 如果右边的极大整块 $r-1$ 满足 `b[r - 1] > y`，那么 $k \leftarrow r-1$。
2. 从节点 k 开始，往下查找，定位到满足 `b[p] > y` 的叶子 p。
3. 在 `a[p - N]` 中查找第一个大于 y 的纵坐标。

我们把 1 2 两步称为**在线段树上二分查找**。

```cpp
int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    int n;
    cin >> n;
    vector<string> type(n);
    vector<int> x(n), y(n);
    for (int i = 0; i < n; i++)
        cin >> type[i] >> x[i] >> y[i];

    vector<int> real_x = compress(x); //坐标压缩
    int N = (int) real_x.size();
    vector<set<int>> a(N);
    vector<int> b(2 * N, -1); //线段树

    for (int i = 0; i < n; i++) {
        if (type[i] == "add") {
            a[x[i]].insert(y[i]);
            int p = x[i] + N;
            b[p] = *a[x[i]].rbegin();
            for (p /= 2; p > 0; p /= 2) {
                b[p] = max(b[p * 2], b[p * 2 + 1]);
            }
        } else if (type[i] == "remove") {
            a[x[i]].erase(y[i]);
            int p = x[i] + N;
            b[p] = a[x[i]].empty() ? -1 : *a[x[i]].rbegin();
            for (p /= 2; p > 0; p /= 2) {
                b[p] = max(b[p * 2], b[p * 2 + 1]);
            }
        } else {
            int l = x[i] + 1 + N, r = N + N;
            int p = -1;
            while (l < r) {
                if (l & 1) {
                    if (b[l] > y[i]) { p = l; break; }
                    l++;
                }
                if (r & 1) {
                    if (b[r - 1] > y[i]) p = r - 1;
                    r--;
                }
                l /= 2; r /= 2;
            }
            if (p == -1)
                cout << -1 << '\n';
            else {
                while (p < N) {
                    if (b[p * 2] > y[i])
                        p = 2 * p;
                    else
                        p = 2 * p + 1;
                }
                cout << real_x[p - N] << ' ' <<
                    *a[p - N].upper_bound(y[i]) << '\n';
            }
        }
    }
}
```

数列分块入门 9

给你一个长为 n 的整数序列 $a = (a_1, \ldots, a_n)$。回答 n 个询问

- `l r`：输出 $a_l, \ldots, a_r$ 的众数。若有多个，输出最小的那个。

限制

- $1 \le n \le 3 \times 10^5$
- a_i 在 int 范围内。

为了突出重点以及便于讲述，以下我们讨论如何求出 $a_l, \ldots, a_r$ 的一个众数，不管它是不是最小的众数。

坐标压缩

设序列 a 的元素的值被压缩为 $[0, N)$。

分块

取 $B = \lfloor \sqrt{n} \rfloor$。
一共有 $\lfloor N/B \rfloor$ 个块。（不是 $\lceil N/B \rceil$ 块，最后剩下的不到 B 项不管。）

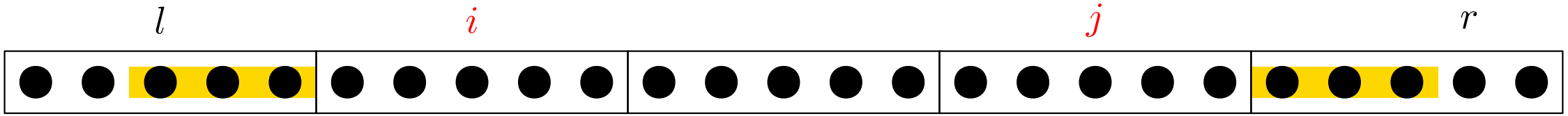
预处理

```
int NB = N / B;
vector<vector<int>> mode(NB, vector<int>(NB));
vector<vector<int>> f(NB, vector<int>(NB));
```

`mode[i][j]`：从第 i 块到第 j 块这一段的众数。

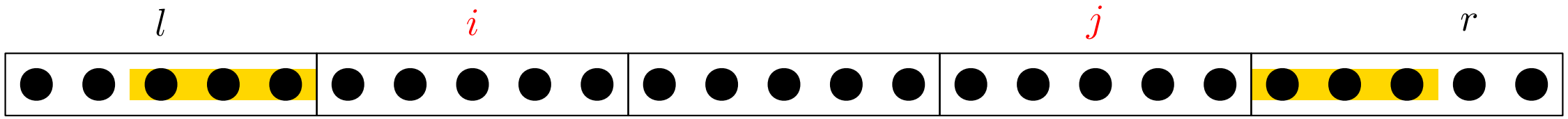
`f[i][j]`：从第 i 块到第 j 块这一段的众数的出现次数。

查询区间 $[l, r)$ 的众数



如果区间 $[l, r)$ 的众数不是 `mode[i][j]`，那么它一定在边块里出现过。

我们检查边块里的数在区间 $[l, r)$ 上出现的次数是不是**更多**。



对于区间 $[l, r)$ 中落在左边块里的一项 a_k，我们检查

- a_k 在区间 $[k, r)$ 上的出现次数是否大于 `f[i][j]`。

为此，我们预先算出一些数据：

```
vector<vector<int>> pos(N);
vector<int> rank(n);
```

- `pos[x]`：x 在序列 a 中出现的位置（$0 \le x < N$）。
- `rank[i]`：a_i 在所有跟它相同的数中排第几？即 i 在 `pos[a[i]]` 里排第几？

例：$a = (2, 3, 3, 0, 1, 1, 4, 5, 1, 4)$，`pos[1]` 是 `{4, 5, 8}`（下标从 0 开始），`rank` 是 `{0, 0, 1, 0, 0, 1, 0, 0, 2, 1}`。

计算 pos 和 rank

```
vector<vector<int>> pos(N);
vector<int> rank(n);
for (int i = 0; i < n; i++) {
    rank[i] = (int) pos[a[i]].size();
    pos[a[i]].push_back(i);
}
```

利用 pos 和 rank

「a_k 在区间 $[k, r)$ 上的出现次数大于 10」可以表述为

```
pos[a[k]].size() > rank[k] + 10 &&
    pos[a[k]][rank[k] + 10] < r
```

检查边块里的数出现次数是否更多

```
int lb = (l + B - 1) / B, rb = r / B;
if (lb > rb) {
    int v = mode[lb][rb - 1];
    int mx = f[lb][rb - 1];
    // 检查左边块里的数
    for (int i = l; i < lb * B; i++) {
        int j = rank[i] + mx;
        while (j < pos[a[i]].size() && pos[a[i]][j] < r)
            j++;
        if (mx < j - rank[i]) {
            mx = j - rank[i];
            v = a[i];
        }
    }
    // 检查右边块里的数
    for (int i = r - 1; i >= rb * B; i--) {
        int j = rank[i] - mx;
        while (j >= 0 && pos[a[i]][j] >= l)
            j--;
        if (mx < rank[i] - j) {
            mx = rank[i] - j;
            v = a[i];
        }
    }
}
```

`j++;` 和 `j--;` 执行的总次数不超过区间 $[l, r)$ 落在左右边块里的部分的长度之和。

完整代码

```cpp
int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    int n; cin >> n;
    vector<int> a(n);
    for (int i = 0; i < n; i++)
        cin >> a[i];

    vector<int> val = compress(a); //坐标压缩
    int N = (int) val.size();

    int B = (int) sqrt(n);
    int NB = n / B;

    vector<vector<pair<int,int>>> mode(NB, vector<pair<int,int>>(NB));

    for (int i = 0; i < NB; i++) {
        int mx = 0, v = -1;
        vector<int> cnt(N);
        for (int j = i; j < NB; j++) {
            for (int k = j * B; k < (j + 1) * B; k++) {
                ++cnt[a[k]];
                if (cnt[a[k]] > mx || (cnt[a[k]] == mx && a[k] < v)) {
                    mx = cnt[a[k]];
                    v = a[k];
                }
            }
            mode[i][j] = {v, mx};
        }
    }

    vector<vector<int>> pos(N);
    vector<int> rank(n);
    for (int i = 0; i < n; i++) {
        rank[i] = (int) pos[a[i]].size();
        pos[a[i]].push_back(i);
    }
```

```cpp
    int q = n;
    while (q--) {
        int l, r;
        cin >> l >> r;
        --l;

        int lb = (l + B - 1) / B, rb = r / B;
        int v = -1, mx = 0;

        if (lb >= rb) {
            for (int i = l; i < r; i++) {
                int j = rank[i] + mx;
                while (j < (int) pos[a[i]].size() && pos[a[i]][j] < r)
                    j++;
                if (mx < j - rank[i]) {
                    mx = j - rank[i];
                    v = a[i];
                } else if (a[i] < v && j - 1 < (int) pos[a[i]].size() && pos[a[i]][j - 1] < r) {
                    v = a[i];
                }
            }
        } else {
            v = mode[lb][rb - 1].first;
            mx = mode[lb][rb - 1].second;
            for (int i = l; i < lb * B; i++) {
                int j = rank[i] + mx;
                while (j < (int) pos[a[i]].size() && pos[a[i]][j] < r)
                    j++;
                if (mx < j - rank[i]) {
                    mx = j - rank[i];
                    v = a[i];
                } else if (a[i] < v && j - 1 < (int) pos[a[i]].size() && pos[a[i]][j - 1] < r) {
                    v = a[i];
                }
            }
            for (int i = rb * B; i < r; i++) {
                int j = rank[i] - mx;
                while (j >= 0 && pos[a[i]][j] >= l)
                    j--;
                if (mx < rank[i] - j) {
                    mx = rank[i] - j;
                    v = a[i];
                } else if (a[i] < v && j + 1 >= 0 && pos[a[i]][j + 1] >= l) {
                    v = a[i];
                }
            }
        }
        cout << val[v] << '\n';
    }
}
```